

Mobile App Backend

Backend API cho ứng dụng mobile sử dụng Express.js với ES Modules

Cấu trúc thư mục

```
Mobile-app/  
├─ app.js           # Tập khởi tạo ứng dụng chính  
├─ package.json     # Các phụ thuộc và cấu hình dự án  
├─ .env.example     # Mẫu biến môi trường  
├─ config/  
│   ├─ database.js  # Cấu hình cơ sở dữ liệu  
│   └─ environment.js # Cấu hình biến môi trường  
├─ middleware/  
│   ├─ corsMiddleware.js # Cấu hình CORS  
│   └─ errorHandler.js  # Xử lý lỗi  
├─ routes/  
│   ├─ index.js      # Route trang chủ  
│   └─ users.js       # Route người dùng  
├─ controllers/  
│   └─ usersController.js  
├─ services/  
│   └─ userService.js  
├─ models/  
│   └─ User.js  
└─ utils/  
    ├─ responseHandler.js # Xử lý phản hồi HTTP  
    └─ logger.js          # Ghi nhật ký
```

Cài đặt

1. Clone hoặc tải dự án

```
cd Mobile-app
```

2. Cài đặt các phụ thuộc

```
npm install
```

3. Tạo file .env

```
cp .env.example .env
```

4. **Cập nhật biến môi trường** trong file `.env` nếu cần thiết

Chạy ứng dụng

Chế độ phát triển (với auto-reload)

```
npm run dev
```

Chế độ sản xuất

```
npm start
```

Ứng dụng sẽ chạy trên `http://localhost:8080`

API Endpoints

Home

- `GET /` - Trang chủ API

Health Check

- `GET /health` - Kiểm tra trạng thái máy chủ

Users

- `GET /api/users` - Lấy tất cả người dùng
- `GET /api/users/:id` - Lấy người dùng theo ID
- `POST /api/users` - Tạo người dùng mới
- `PUT /api/users/:id` - Cập nhật người dùng
- `DELETE /api/users/:id` - Xóa người dùng

Cấu trúc Request/Response

Request Body (POST/PUT)

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "phone": "0123456789"
}
```

Response thành công (200, 201)

```
{
  "success": true,
  "message": "Success message",
  "data": {},
  "timestamp": "2024-04-17T10:00:00.000Z"
}
```

Response lỗi (4xx, 5xx)

```
{
  "success": false,
  "message": "Error message",
  "error": "Error details",
  "timestamp": "2024-04-17T10:00:00.000Z"
}
```

Mô tả các thư mục

config/

Chứa các file cấu hình cho:

- Kết nối cơ sở dữ liệu
- Biến môi trường

middleware/

Chứa các middleware Express:

- CORS middleware
- Error handler middleware
- Middleware tùy chỉnh khác

routes/

Định nghĩa các route của API và gắn với controller

controllers/

Xử lý logic các request, gọi service và trả về response

services/

Chứa logic kinh doanh, xử lý dữ liệu, tương tác với cơ sở dữ liệu

models/

Định nghĩa các mô hình và schema dữ liệu

utils/

Hàm tiện ích:

- `responseHandler.js` - Hàm format response chuẩn
- `logger.js` - Ghi nhật ký

Sử dụng ES Modules

Dự án này sử dụng ES Modules (import/export) thay vì CommonJS.

Ví dụ:

```
// Import
import express from "express";
import { getAllUsers } from "../services/usersService.js";

// Export default
export default router;

// Export named
export const getAllUsers = async () => {
  // ...
};
```

Mở rộng dự án

Thêm route mới

1. Tạo controller mới trong `controllers/`
2. Tạo service mới trong `services/` (tuỳ chọn)
3. Tạo file route mới trong `routes/`
4. Import và sử dụng route trong `app.js`

Thêm middleware

1. Tạo middleware trong thư mục `middleware/`
2. Import trong `app.js`
3. Sử dụng `app.use(middleware)`

Kết nối cơ sở dữ liệu

Cập nhật `config/database.js` với cấu hình:

- Thêm MongoDB driver: `npm install mongoose` hoặc `npm install mongodb`
- Kết nối trong file khởi tạo ứng dụng

Biến môi trường

Xem file `.env.example` để biết các biến cần thiết:

- `NODE_ENV` - Môi trường (development/production)
- `PORT` - Cổng chạy server
- `FRONTEND_URL` - URL frontend (để cấu hình CORS)
- `DATABASE_URL` - URL kết nối cơ sở dữ liệu
- `JWT_SECRET` - Secret key cho JWT

Logging

Sử dụng hàm `logger` để ghi nhật ký:

```
import { logger } from "../utils/logger.js";

logger.info("Info message");
logger.warn("Warning message");
logger.error("Error message");
logger.debug("Debug message");
```

Sentry

Backend đã được cấu hình với Sentry Node SDK qua file `instrument.js` để khởi tạo sớm nhất có thể.

Biến môi trường cần thêm

- `SENTRY_DSN`
- `SENTRY_TRACES_SAMPLE_RATE`
- `SENTRY_ENABLE_DEBUG_ENDPOINT` để bật endpoint test trong môi trường không production

Kiểm tra nhanh

- `GET /debug/sentry/message` để gửi test message
- `GET /debug/sentry/error` để tạo lỗi thử

Checklist xác nhận

1. Điền `SENTRY_DSN` trong file `.env`.
2. Giữ `SENTRY_ENABLE_DEBUG_ENDPOINT=1` chỉ khi test ở local/dev.
3. Mở backend và gọi `GET /debug/sentry/message` rồi kiểm tra event trong Sentry.
4. Gọi `GET /debug/sentry/error` để xác nhận error capture.
5. Sau khi kiểm tra xong, tắt `SENTRY_ENABLE_DEBUG_ENDPOINT` về 0.
6. Khi deploy phiên bản mới, cập nhật `SENTRY_RELEASE` để theo dõi lỗi theo release.

Upload Source Maps

Nếu bạn muốn stack trace dễ đọc hơn, dùng Sentry wizard để cấu hình sourcemaps:

```
npx @sentry/wizard@latest -i sourcemaps --saas --org <your-org> --project  
<your-project>
```

Sau đó cấu hình theo hướng dẫn của wizard để upload sourcemaps trong pipeline build.

Xử lý lỗi

Lỗi được xử lý tập trung qua middleware `errorHandler`:

```
app.use(errorHandler);
```

Các lỗi sẽ được ghi nhật ký và trả về cho client dưới định dạng chuẩn.

Hỗ trợ

Nếu gặp vấn đề, vui lòng kiểm tra:

- File `.env` đã được cấu hình đúng không?
- Tất cả package đã được cài đặt không? (`npm install`)
- Cổng 8080 có bị sử dụng không?

Happy coding! 🚀